LẬP TRÌNH WEB

CHƯƠNG 5

XÂY DỰNG REST API

Giảng viên: TS. Đặng Trung Thành

Chuẩn đầu ra của chương

- Lý thuyết
  - Giải thích được REST API là gì và tính năng của Django REST framework (DRF)
  - Sử dụng được các thư viện của DRF để tuần tự hóa dữ liệu
  - Sử dụng được Viewset và router để tổ chức và ánh xạ URL cho các API
  - Giải thích được tiến trình xác thực người dùng dựa trên token
- Thực hành
  - Tạo được các REST API có tuần tự hóa dữ liệu
  - Sử dụng được Viewset và router để tổ chức các API
  - Triển khai được xác thực người dùng dựa trên token

Nội dung

- Giới thiệu về REST API và Django REST framework
- Tuần tự hóa dữ liệu
- Đơn giản hóa mã nguồn sử dụng ViewSet
- Triển khai xác thực

5.1 Giới thiệu về REST API và Django REST framework

API trong các ứng dụng web hiện đại

- Các ứng dụng web hiện đại thường có phần frontend được xây dựng bằng các thư viện riêng, như ReactJS hay AngularJS
  - Cho phép xây dựng các giao diện người dùng động
  - Không giao tiếp trực tiếp với CSDL hay mã nguồn của trình chủ

  →Sử dụng **REST API**
- API (web) được dùng để thúc đẩy sự tương tác giữa các thành phần phần mềm khác nhau sử dụng giao thức HTTP
  - Sử dụng các thông điệp request và response để giao tiếp
- Trong Django: sử dụng **Django REST framework (DRF)**

5.1 Giới thiệu về REST API và Django REST framework

REST API

- Được dùng phổ biến nhất trong ứng dụng web hiện đại
- Là một loại API tập trung vào giao tiếp và đồng bộ trạng thái của các đối tượng giữa máy chủ CSDL và trình khách ở frontend
- Sử dụng kiến truc tách rời, tạo sự linh hoạt và dễ cập nhật, thay đổi cho các thành phần

  → Các ứng dụng khách khác nhau có thể dùng chung API
- Là giao thức không trạng thái (stateless)
  - Các yêu cầu sẽ được xử lý và kết quả được phản hồi ngay, không lưu lại dữ liệu trung gian
- **Dịch vụ web RESTful (RESTful web service):** Tập các REST API thực hiện một tập các tác vụ

5.1 Giới thiệu về REST API và Django REST framework

Django REST framework (DRF)

- Là một thư viện nguồn mở trên Python dùng để phát triển các REST API
- Cài đặt và cấu hình
  - Cài đặt:

```
pip install djangorestframework
```

  - Khai báo sử dụng:

```
INSTALLED_APPS = [
    ...
    "rest_framework",
]
```

- Khai báo API cho view dạng hàm

```
from rest_framework.decorators import api_view

@api_view
def my_view(request):
    ...
```

5.1 Giới thiệu về REST API và Django REST framework

Django REST framework (DRF)

- Khai báo API cho view dạng lớp
  - Kế thừa từ các lớp APIView cung cấp bởi DRF
    - + CreateAPIView: dùng trong tạo mới các đối tượng model
    - + ListAPIView: dùng để đọc ra danh sách các đối tượng model
    - + RetrieveAPIView: dùng để đọc ra một đối tượng model
    - + DestroyAPIView: dùng trong xóa một đối tượng model
    - + ... *(Danh sách đầy đủ: https://www.django-rest-framework.org/api-guide/generic-views/)*

Ví dụ:

```
class AllBooks(ListAPIView):
    queryset = Book.objects.all()
    serializer_class = BookSerializer
```

5.2 Tuần tự hóa dữ liệu

- API Sử dụng định dạng JSON
- Sử dụng lớp serializers.Serializer của DRF để chuyển đối tượng trên Python thành dạng JSON và ngược lại

Ví dụ:

```
# Khai báo lớp PublisherSerializer
from rest_framework import serializers
class BookSerializer(serializers.Serializer):
    title = serializers.CharField()
    publication_date = serializers.DateField()
# Khai báo API view
@api_view()
def all_books(request):
    books = Book.objects.all()
    book_serializer = BookSerializer(books, many=True)
    return Response(book_serializer.data)
```

5.2 Tuần tự hóa dữ liệu

- **Sử dụng ModelSerializer**
  - Liên kết trực tiếp với các lớp Model
  - Tận dụng các khai báo cho các thuộc tính đã có trong Model

Ví dụ:

```
from rest_framework import serializers
from .models import Contributor

class ContributorSerializer(serializers.ModelSerializer):

    class Meta:
        model = Contributor
        fields = [‘first_names', 'last_names', 'email']
```

5.3 Đơn giản hóa mã nguồn sử dụng ViewSet

Giới thiệu ViewSet

- Viewset: là tập các view được biểu diễn trong 1 lớp

  Ví dụ: *Viewset kết hợp view (dạng lớp) AllBooks và BookDetail thành một lớp duy nhất*

- Lớp ModelViewSet: cung cấp các thao tác liệt kê, đọc, thêm, cập nhật và xóa đối tượng trên các đối tượng model

  Ví dụ:

```
class UserViewSet(viewsets.ModelViewSet):
    serializer_class = UserSerializer
    queryset = User
```

- Lớp ReadOnlyModelViewSet: dạng đơn giản hơn của lớp ModelViewSet. Chỉ cho phép liệt kê và truy cập các đối tượng model.

5.3 Đơn giản hóa mã nguồn sử dụng ViewSet

Cấu hình URL sử dụng router và viewset

- Router: khi sử dụng cùng một viewset, tự động tạo URL của các điểm cuối (lời gọi API) cho viewset

Ví dụ: *Khai báo router cho viewset UserViewSet*

```
from rest_framework import routers
router = routers.SimpleRouter()
router.register(r'users', UserViewSet)
urlpatterns = router.urls
```

Khai báo này tạo ra các ánh xạ URL khác nhau để thực hiện các thao tác trên đối tượng model:

- *URL:* `/api/users/` *: để liệt kê các user*
- *URL:* `/api/users/123` *: để xem thông tin chi tiết một user*

5.4 Triển khai xác thực

- Việc xác thực cho phép giới hạn người dùng cụ thể được phép sử dụng API
- Các hình thức xác thực:
  - Cơ bản (BasicAuthentication)
    - Sử dụng username và mật khẩu, chỉ dùng trong giai đoạn kiểm thử
    - Khi xác thực thành công trong đối tượng request sẽ có thuộc tính user
  - Phiên (SessionAuthentication)
    - Tương tự BasicAuthentication, phù hợp với trình khách giao tiếp bằng AJAX (cùng chia sẻ thông tin phiên khi truy cập các trang web của website)
  - Token (TokenAuthentication)
    - Sử dụng token thay cho username và mật khẩu
    - Token tương ứng với mỗi người dùng được lưu trong CSDL
    - Token được sử dụng để xác thực trong mỗi lời gọi API

(Tham khảo thêm: https://www.django-rest-framework.org/api-guide/authentication/)

Hết chương 5